Programming Fundamentals

Dafny - week 8

A. Meijster

University of Groningen

university of groningen

While loop

Like C, Dafny has a `while` loop.

Its syntax is practically the same, except that (like `if`) you are not required to place the guard between parentheses.

So, a while loop looks like:

```
while guard
{
  statements;
}
```

university of groningen

Specification of While loops

With the Hoare-triple $\{P\}$ **while** B **do** S **end** $\{Q\}$ we denote a **while**-program fragment that establishes a state in which *postcondition* Q holds if the program was started in a state in which precondition P holds.

university of groningen

Semantics of $\{P\}$ while B do S end $\{Q\}$

Let us assume that the loop terminates correctly. The loop continues, as long as the *guard* B holds. Clearly, after termination of the loop $\neg B$ holds.

Let us also assume that S maintains some predicate J. We say that J is an *invariant*:

$$\{J\}\ S\ \{J\}$$

It follows that after termination of the loop $J \wedge \neg B$ holds.

Apparently, we need: $J \wedge \neg B \Rightarrow Q$

Besides, we need to make sure that J holds, just before the start of the loop.
So, we may need some initialization S_0 such that: $\{P\}\ S_0\ \{J\}$

university of groningen

termination of a while-loop

Clearly, a **while**-command is only correct if it terminates.

How can we proof, that a loop terminates?

Let vf be some function that maps program states to $\mathbb{Z}$. Now assume that vf $= K$ (for some $K \in \mathbb{N}$) just before the loop starts.

Let us also assume that the value of vf decreases after every execution of S, so for all $V \in \mathbb{Z}$: $\{\text{vf} = V\}\ S\ \{vf < V\}$

Let us also assume that $J \wedge B \Rightarrow \text{vf} \geq 0$.

Then, the loop will surely terminate (within K iterations)!

university of groningen

Proof rule: while-loop

This informal analysis leads to the following formal proof rule:

From

- vf $\in \mathbb{Z}$
- $J \wedge B \Rightarrow \text{vf} \geq 0$
- $\{J \wedge B\}\ S\ \{J\}$
- $\{J \wedge B \wedge \text{vf} = V\}\ S\ \{\text{vf} < V\}$

we may conclude

$$\{J\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{end}\ \{J \wedge \neg B\}$$

university of groningen

Proof rule(equivalent): while-loop

From

- $\text{vf} \in \mathbb{Z}$
- $J \wedge B \Rightarrow \text{vf} \geq 0$
- $\{J \wedge B \wedge \text{vf} = V\}\ S\ \{J \wedge \text{vf} < V\}$

we may conclude

$$\{J\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{end}\ \{J \wedge \neg B\}$$

university of groningen

While loops: Proof rule in Dafny

In Dafny, we decorate loops with an invariant using the keyword `invariant`.
Moreover, the variant function can be specified using the keyword `decreases`.
The most general form is:

```
while B
invariant J
decreases vf
{
  Body
}
```

Often, the `decrease` clause can be omitted, because Dafny guesses a suitable variant function.
Typically, a loop like

```
while i < n
invariant 0 <= i <= n && ...
{
  doSomething;
  i := i + 1;
}
```

does not need a `decreases` clause, because Dafny 'guesses' vf $= n - i$.

university of groningen

Step-by-step design of a while-program: $\{P\}\ T\{Q\}$

1. Choose an invariant J and a guard B such that $J \wedge \neg B \Rightarrow Q$
2. Initialization: Find a command T_0 such that $\{P\}\ T_0\ \{J\}$.
   Note that if $P \Rightarrow J$, then this step can be skipped.
3. Variant function: choose a vf $\in \mathbb{Z}$ and prove $J \wedge B \Rightarrow \text{vf} \geq 0$
4. Body of the loop: Find a command S such that
   $\{J \wedge B \wedge \text{vf} = V\}\ S\ \{J \wedge \text{vf} < V\}$
5. Conclude that $\{P\}\ T_0;$ **while** B **do** S **end** $\{Q\}$

university of groningen

While loops: an almost trivial example

```
method Identity(n: int) returns (i: int)
requires n >= 0
ensures i == n
{
  i := 0;
  while i < n
  {
    i := i + 1;
  }
}
```

If we feed the Dafny verifier with this code fragment it responds with:

Dafny reports

```
Error:  the postcondition could not be proved:  ensures i == n
```

To help Dafny prove the correctness, we need to supply it with an invariant.
We choose the invariant: $J:\ 0 \leq i \leq n$

With this extra information, Dafny is able to verify the correctness.

university of groningen

While loops: an almost trivial example

```
method Identity(n: int) returns (i: int)
requires n >= 0
ensures i == n
{
  i := 0;
  while i < n
  invariant 0 <= i <= n
  {
    i := i + 1;
  }
}
```

We follow the roadmap to see which proofs Dafny made for us.

1. Choose an invariant J and a guard B such that $J \wedge \neg B \Rightarrow Q$.
   From $J : 0 \leq i \leq n$ and $i \geq n$ we can conclude that $i = n$. Therefore we choose:
   $$J : \ 0 \leq i \leq n$$
   $$B : \ i < n$$
   So, the proof rule $J \wedge \neg B \Rightarrow Q$ is satisfied.
2. Initialization: We need to show that $\{P\}\ T_0\ \{J\}$
   In this case, that is trivial. It is clear that $\{n \geq 0\}\ i := 0;\ \{0 \leq i \leq n\}$

university of groningen

While loops: an almost trivial example

```
method Identity(n: int) returns (i: int)
requires n >= 0
ensures i == n
{
  i := 0;
  while i < n
  invariant 0 <= i <= n
  {
    i := i + 1;
  }
}
```

3. Variant function: choose a vf $\in \mathbb{Z}$ and prove $J \wedge B \Rightarrow \text{vf} \geq 0$
   We do not really need to choose a variant function, because Dafny chooses the default `decreases n-i` clause for us.
   The invariant says that $i \leq n$, so clearly $\text{vf} = n - i \geq 0$.

university of groningen

While loops: an almost trivial example

```
method Identity(n: int) returns (i: int)
requires n >= 0
ensures i == n
{
  i := 0;
  while i < n
  invariant 0 <= i <= n
  {
    i := i + 1;
  }
}
```

4 Body of the loop: $\{J \wedge B \wedge \text{vf} = V\}\ i := i + 1;\ \{J \wedge \text{vf} < V\}$

$\{J \wedge B \wedge \text{vf} = V\}$
$(*$ definitions J, B, and vf $*)$
$\{0 \leq i \leq n \wedge i < n \wedge \ \wedge n - i = V\}$
$(*$ calculus; prepare $i := i + 1$ $*)$
$\{0 \leq i + 1 \leq n \wedge \ \wedge n - (i + 1) < V\}$
$i := i + 1;$
$\{0 \leq i \leq n \wedge n - i < V\}$
$(*$ definitions J and vf $*)$
$\{J \wedge \text{vf} < V\}$

university of groningen

Summing integers in Dafny: $\texttt{sum(n)} = \Sigma_{i=0}^{n} i$

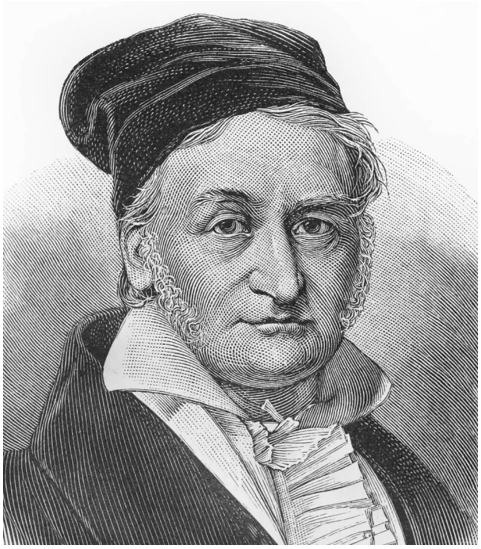
```
ghost function sum(n: nat): int
{
  if n == 0 then 0 else n + sum(n - 1)
}

method Sum(n: nat) returns (s: int)
ensures s == sum(n)
{
  s := 0;
  var i := 0;
  while i < n
  invariant 0<=i<=n && s == sum(i)
  {
    i := i + 1;
    s := s + i;
  }
}
```

Dafny reports

```
Dafny program verifier finished with 3 verified, 0 errors
```

Gauss summing integers: $\texttt{sum(n)} = \Sigma_{i=0}^{n} i$

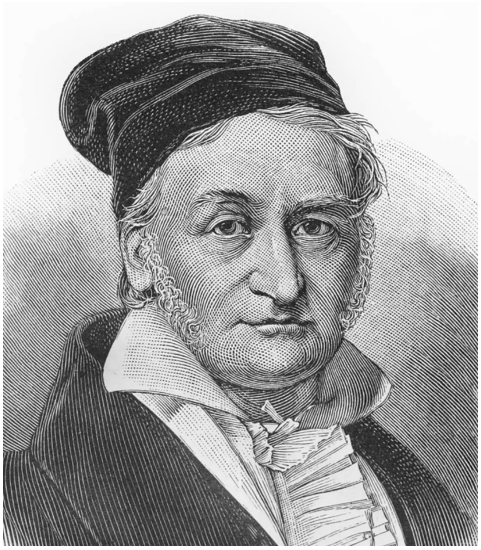


Carl Friedrich Gauss
(1777-1855)

A famous story is a tale from when Gauss was still at primary school. One day Gauss' teacher asked his class to add together all the numbers from 1 to 100, assuming that this task would occupy them for quite a while. He was shocked when young Gauss, after a few seconds thought, wrote down the answer 5050.

university of groningen

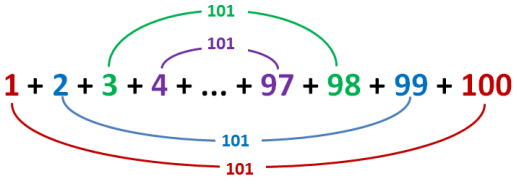
Gauss summing integers: $\texttt{sum(n)} = \Sigma_{i=0}^{n} i$



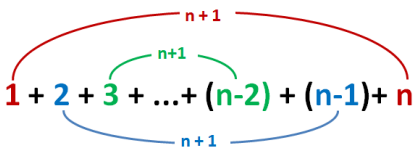
Carl Friedrich Gauss
(1777-1855)

How did Gauss do this?

He simply realized the following:



Or, more generally:



Conclusion: $\sum_{i=0}^{n} i = \frac{n(n+1)}{2}$

university of groningen

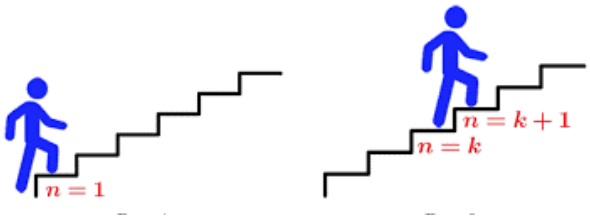
Formal proof of Gauss' identity: Mathematical induction

We can prove Gauss' identity uses the *principal of mathematical induction*. The technique works as follows:

Let $P(n)$ be some predicate which depends on the natural number n.
To prove that $P(n)$ holds for all natural numbers n it suffices to prove:

- $P(0)$: we call this the base case.
- $P(n) \Rightarrow P(n+1)$ for all n: we call this the inductive step





In formal notation: $P(0) \wedge (\forall n :: P(n) \Rightarrow P(n+1)) \quad \Rightarrow \quad (\forall n :: P(n))$

university of groningen

Formal proof of Gauss' identity: Mathematical induction

To prove Gauss's identity using induction, we define $P(n) \equiv \sum_{i=0}^{n} i = \frac{n(n+1)}{2}$.

$P(0)$ holds trivially: $P(0) = \sum_{i=0}^{0} i = 0 = \frac{0(0+1)}{2}$

Next, we need to show that $P(n) \Rightarrow P(n+1)$:

$$
\begin{array}{ll}
 & P(n+1) \\
\equiv & \\
 & \sum_{i=0}^{n+1} i = \frac{(n+1)(n+2)}{2} \\
\equiv & \\
 & n+1+\sum_{i=0}^{n} i = \frac{(n+1)(n+2)}{2} \\
\equiv & \\
 & n+1+\sum_{i=0}^{n} i = \frac{n(n+1)}{2} + n + 1 \\
\equiv & \\
 & \sum_{i=0}^{n} i = \frac{n(n+1)}{2} \\
\Leftarrow & \{\text{actually } \equiv \text{ but only } \Leftarrow \text{ is needed}\} \\
 & P(n) \qquad\qquad \text{QED}
\end{array}
$$

university of groningen

Gauss and Dafny summing integers: $\texttt{sum(n)} = \Sigma_{i=0}^{n} i$

But, we can also prove the correctness of Gauss' theorem using Dafny.

The only thing we need to do is strengthen the invariant, and let Dafny do the proof:

```
method Sum(n: nat) returns (s: int)
ensures s == sum(n)
{
  s := 0;
  var i := 0;
  while i < n
  invariant 0<=i<=n && s == sum(i) && s == i*(i + 1)/2
  {
    i := i + 1;
    s := s + i;
  }
}
```

Dafny reports

```
Dafny program verifier finished with 3 verified, 0 errors
```

The loops terminates with `i==n`, so `sum(n)==n*(n+1)/2`.

QED.

university of groningen

Gauss and Dafny summing integers: $\texttt{sum(n)} = \Sigma_{i=0}^{n} i$

Now, you may feel a bit fooled here,.....
But, actually we did a proof by mathematical induction using Dafny.

To see this, let us prove that `s==i*(i+1)/2 ==>` $\mathcal{WP}$`(i:=i+1;s:=s+i, s==i*(i+1)/2)`.

```
  wp(i:=i+1;s:=s+i, s==i*(i + 1)/2)
=
  wp(i:=i+1, wp(s:=s+i, s==i*(i + 1)/2))
=
  wp(i:=i+1, s+i==i*(i + 1)/2)
=
  s + i + 1==(i + 1)*(i + 2)/2
=
  s + i + 1==i*(i + 1)/2 + i + 1
<==
  s==i*(i + 1)/2
```

Hence, the proof rule $\{J \wedge B\}$ *Body* $\{J\}$ is actually the inductive step in disguise.
Moreover, the initialization of the invariant is the base case. We conclude that the correctness proof of a while-loop corrresponds with a proof by mathematical induction.

university of groningen

Induction via while-loops

In fact, the following template can be used for inductive proofs.

```
ghost predicate toProve(n: nat)
{
  // put here your property P(n)
}

method InductionProof(n: nat)
ensures toProve(n)
{
  var i := 0;
  while i < n
  invariant 0<=i<=n && toProve(i)
  {
    i := i + 1;
  }
}
```

university of groningen

Induction via while-loops

So, we can rewrite the program that proves Gauss' identity to:

```
ghost function sum(n: nat): int
{
  if n == 0 then 0 else n + sum(n - 1)
}

ghost predicate toProve(n: nat)
{
  sum(n) == n*(n+1)/2
}

method InductionProof(n: nat)
ensures toProve(n)
{
  var i := 0;
  while i < n
  invariant 0<=i<=n && toProve(i)
  {
    i := i + 1;
  }
}
```

university of groningen

Another example: integer division in Dafny

We will implement integer division and remainder using only addition and subtraction.

```
method QuotRem(a : nat, d : nat) returns (q : nat, r : nat)
requires d > 0
ensures a == q*d + r && 0<=r<d
{
  q, r := 0, a;
  while d <= r
  invariant  0 <= r && a == q*d + r
  decreases  r - d
  {
    q := q + 1;
    r := r - d;
  }
}
```

university of groningen

Example: Step-by-step design of a while-program

Consider the following specification:

const $x : \mathbb{R}$;
var $y : \mathbb{R}$, $n : \mathbb{Z}$;
$\{P : n \geq 0 \wedge x^n = Y\}$
S
{Q: y=Y}

What does S compute? Answer: x^n, i.e. exponentiation.

0. We decide that we need a **while**-program: we assume that there is no function **pow** available.
1. Choose an invariant J and a guard B such that $J \wedge \neg B \Rightarrow Q$.
   From $y \cdot x^n = Y$ and $n = 0$ we can conclude that $y = Y$. Therefore we choose:

$$J : n \geq 0 \wedge y \cdot x^n = Y$$

$$B : n \neq 0$$

Clearly, we have:
$J \wedge \neg B \quad \Rightarrow \quad n = 0 \wedge y \cdot x^n = Y \quad \Rightarrow \quad y = Y \quad \equiv \quad Q$

university of groningen

Continued: Step-by-step design of a while-program

2 Initialization: Find a command T_0 such that $\{P\}\ T_0\ \{J\}$

$P : n \geq 0\ \wedge\ x^n = Y$
$J : n \geq 0\ \wedge\ y \cdot x^n = Y$
If $y = 1$, then the predicates P and J are equivalent:

$\{P :\ n \geq 0\ \wedge\ x^n = Y\}$
$\quad (*$ calculus; prepare $y := 1\ *)$
$\{n \geq 0\ \wedge\ 1 \cdot x^n = Y\}$
$y := 1;$
$\{J :\ n \geq 0\ \wedge\ y \cdot x^n = Y\}$

3 Variant function: choose a vf $\in \mathbb{Z}$ and prove $J\ \wedge\ B \Rightarrow \text{vf} \geq 0$
Since J says that $n \geq 0$, we can simply choose vf $= n$.
Clearly, we have: $J\ \wedge\ B \Rightarrow J \Rightarrow n \geq 0 \equiv vf \geq 0$

university of groningen

Continued: Step-by-step design of a while-program

4 Body of the loop: Find a command S such that
$\{J\ \wedge\ B\ \wedge\ \text{vf} = V\}\ S\ \{J\ \wedge\ \text{vf} < V\}$

Since vf must decrease, it is natural to expect that we need:
$n := n - 1$.

$\{J\ \wedge\ B\ \wedge\ \text{vf} = V\}$
$\quad (*$ definitions J, B, and vf $*)$
$\{n \geq 0\ \wedge\ y \cdot x^n = Y\ \wedge\ n \neq 0\ \wedge\ n = V\}$
$\quad (*$ calculus; prepare $n := n - 1\ *)$
$\{n - 1 \geq 0\ \wedge\ y \cdot x \cdot x^{n-1} = Y\ \wedge\ n - 1 < V\}$
$y := y * x;$
$\{n - 1 \geq 0\ \wedge\ y \cdot x^{n-1} = Y\ \wedge\ n - 1 < V\}$
$n := n - 1;$
$\{n \geq 0\ \wedge\ y \cdot x^n = Y\ \wedge\ n < V\}$
$\quad (*$ definitions J and vf $*)$
$\{J\ \wedge\ \text{vf} < V\}$

university of groningen

Continued: Step-by-step design of a while-program

5 Conclude that $\{P\}\ T_0;$ **while** B **do** S **end** $\{Q\}$ solves the problem.

We found the following program fragment:

$$
\begin{array}{l}
\quad \{P:\ n \geq 0 \ \wedge\ x^n = Y\} \\
y := 1; \\
\quad \{J: n \geq 0 \ \wedge\ y \cdot x^n = Y\} \\
\qquad (* \ \mathrm{vf} = n \ *) \\
\textbf{while } n \neq 0 \textbf{ do} \\
\quad y := y * x; \\
\quad n := n - 1; \\
\textbf{end}; \\
\quad \{Q:\ y = Y\}
\end{array}
$$

university of groningen

Exponentiation in Dafny

```
ghost function Pow(a: nat, e: nat): nat
{ // returns a raised to the power e
  if e == 0 then 1 else a*Pow(a, e-1)
}

method Power(x: nat, e: nat) returns (y: nat)
ensures  y == Pow(x, e)
{ // note: Pow(x,e) plays the role of Y
  y := 1;
  var n := e;
  while n != 0
  invariant 0<=n &&  y*Pow(x,n) == Pow(x,e)
  decreases n
  {
    y := y*x;
    n := n - 1;
  }
}
```

university of groningen

Example: Powers of 2

Compute the smallest power of 2 greater than a given natural number $x > 0$:

const $x : \mathbb{Z}$;
var $i, y : \mathbb{Z}$;
$\{P : x > 0\}$
T
$\{Q : y = 2^i \wedge x < y \leq 2 \cdot x\}$

1 Choose an invariant J and a guard B such that $J \wedge \neg B \Rightarrow Q$.
For the invariant we choose a part of Q.

$$J : y = 2^i \wedge y \leq 2 \cdot x$$

For B we choose $\neg(x < y)$, so $B : y \leq x$

Clearly, we have: $J \wedge \neg B \equiv Q$

university of groningen

Continued: Powers of 2

2 Initialization: $\{P : x > 0\}\ T_0\ \{J : y = 2^i \wedge y \leq 2 \cdot x\}$

$\{x > 0\}$
$(*$ calculus $*)$
$\{1 = 2^0 \wedge 1 \leq 2 \cdot x\}$
$i := 0;$
$\{1 = 2^i \wedge 1 \leq 2 \cdot x\}$
$y := 1;$
$\{y = 2^i \wedge y \leq 2 \cdot x\}$

3 Variant function: choose a vf $\in \mathbb{Z}$ such that $J \wedge B \Rightarrow \text{vf} \geq 0$
$B : y \leq x$, so the program terminates if $x - y < 0$.
We choose $\text{vf} = x - y \in \mathbb{Z}$.

Since $B \equiv \text{vf} \geq 0$, it is clear that $J \wedge B \Rightarrow vf \geq 0$

university of groningen

Continued: Powers of 2

4 Body of the loop: Find a command S such that
$\{J \wedge B \wedge \text{vf} = V\}\ S\ \{J \wedge \text{vf} < V\}$

$\{J \wedge B \wedge \text{vf} = V\}$
$(*$ definitions J, B, and vf $*)$
$\{y = 2^i \wedge y \leq 2 \cdot x \wedge y \leq x \wedge x - y = V\}$
$(*$ calculus; logic; $y = 2^i \Rightarrow y > 0 \Rightarrow 2 \cdot y > 0 *)$
$\{2 \cdot y = 2^{i+1} \wedge 2 \cdot y \leq 2 \cdot x \wedge x - 2 \cdot y < V\}$
$y := 2 * y;$
$\{y = 2^{i+1} \wedge y \leq 2 \cdot x \wedge x - y < V\}$
$i := i + 1;$
$\{y = 2^i \wedge y \leq 2 \cdot x \wedge x - y < V\}$
$(*$ definitions J and vf $*)$
$\{J \wedge \text{vf} < V\}$

university of groningen

Continued: Powers of 2

5 Conclusion: the following program fragment satisfies the specification:

$\{P: x > 0\}$
$i := 0;$
$y := 1;$
$\{J: y = 2^i \wedge y \leq 2 \cdot x\}$
$(*$ vf $= x - y *)$
while $y \leq x$ **do**
$y := 2 * y;$
$i := i + 1;$
end;
$\{Q: y = 2^i \wedge x < y \leq 2 \cdot x\}$

university of groningen

Continued: Powers of 2

We derived manually a correct program fragment. Let's try it again with Dafny. We start by trying to specify the method `GTpow2`.

```
method GTpow2(x: nat) returns (y: nat)
requires x > 0
ensures x < y <= 2*x && y == Power(2,i)
```

However, this fails. The verfier tells us:

Error message

```
Error:  unresolved identifier:  i
```

How to solve this?

We only needed `i` to specify that `y`$==2^{\texttt{i}}$, so $\exists i :: y = 2^i$ suffices.

university of groningen

Continued: Powers of 2

That observation helps. Dafny now accepts the following program:

```
ghost function Power(a: nat, e: nat): nat {
  if e == 0 then 1 else a*Power(a, e-1)
}

method GTpow2(x: nat) returns (y: nat)
requires x > 0
ensures x < y <= 2*x && (exists i:: y == Power(2,i))
{
  var i := 0;
  y := 1;
  while y <= x
  invariant y <= 2*x && y == Power(2,i)
  {
    y := 2*y;
    i := i + 1;
  }
}

method Main()
{
  var p42 := GTpow2(42);
  print "smallest power of 2 greater than 42 is ", p42, ".\n";
}
```

university of groningen